2012 IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum

FPGA Implementation of SRAM-based Ternary Content Addressable Memory

Zahid Ullah[1], Manish Kumar Jaiswal[2], Y.C. Chan[3], and Ray C.C. Cheung[4]
{[1]*zullah2,*[2]*mkjaiswal2*}*@student.cityu.edu.hk,* {[3]*eeycchan,*[4]*r.cheung*}*@cityu.edu.hk*
Department of Electronic Engineering
City Univeristy of Hong Kong
Kowloon Tong, Hong Kong

***Abstract*—Content Addressable Memory (CAM) is a special memory that accomplishes search operation in a single clock cycle but CAM has disadvantages like low bit density and high cost per bit. In this paper, we present an implementation of a 512 x 36 SRAM-based TCAM (SR-TCAM) on a Virtex-5 FPGA, which is the strength of SR-TCAM because currently classical TCAMs cannot be implemented on FPGA. We have used two synthesis optimizations (BRAM-AUTO and BRAM = BLOCK_POWER2) using BRAMs on FPGA. Thus, user can choose design parameters that are suitable for his application. The power data has been measured using Xilinx X-Power by using activities for search operations of SR-TCAM. We have computed power consumption by taking the average of 1000 search operations with 100 MHz clock speed to have better power estimation, which results, for one of the design, in an average power consumption of 2.11 mW. SR-TCAM exploits dense SRAM and achieves comparable search performance in two clock cycles. Thus, SR-TCAM is a feasible and practical alternative to traditional CAMs.**

***Keywords*-Content Addressable Memroy (CAM); Ternary Content Addressable Memory (TCAM); Potential Matching Address (PMA); Matching Address (MA); Vertical Partitioning (VP); SRAM; Power Consumption; FPGA; Latency**

I. Introduction

Content Addressable Memory (CAM) compares input data against the stored data in parallel and outputs the address of the matched data. CAMs can find their applications in network routers, artificial-intelligence, translation look-aside buffers in microprocessors, cache memory, data compression, and radar signal tracking. Recent applications include real-time pattern matching in virus-detection and intrusion-detection systems, gene pattern searching in bioinformatics, and image processing [1].

CAM is an outgrowth of the RAM technology but the searching function is quite opposite for both of them. RAM needs an address to output the required data, whereas to CAM, contents are provided and matching address is received at the output.

CAM is popular for its high speed search operation. However, the speed of CAM comes at the cost of high power consumption, low bit density, and high cost per bit [2]. Ternary CAMs (TCAMs) cost is about 30 times more per bit of storage than DDR SRAM [3]. In addition, TCAM consumes 150 times more power per bit than SRAM [4]. High-power consumption increases junction temperature, which increases leakage currents, reduces chip performance, and degrades reliability of CAM devices [1].

Pattern capacity in CAM devices is very limited. CAM technology does not evolve as fast as the RAM does. RAM technology is driven by many applications, particularly computers and consumer electronic products, and hence cost per bit is continuously decreasing, as opposed to CAM technology [5]. RAMs appear in a wider variety of sizes and flavours. It is more generic and widely available and enables to avoid the heavy licensing and royalty costs charged by some CAM vendors [6]. Table I indicates that SRAM outperforms TCAM in bit density, speed, and power consumption.

Table I
Comparison between TCAM and SRAM

Parameters	TCAM (18 M bits Chip)	SRAM (18 M bits Chip)
Maximum Clock Rate (MHz)	200 [7]	400 [15], [16]
Number of Transistors in a Cell	16	6
Power Consumption (Watts)	12∼15 [8]	≈ 0.1[9]

Hence, there is a need of an innovative TCAM design that capitalizes dense SRAM and achieves advantages like bit density, lower cost, and comparable search performance.

This paper aims at presenting such an alternative solution. It emulates TCAM functionality with SRAM memory. Recently, such an SRAM-based architecture has been proposed in [10] and [17]. We call this architecture SR-TCAM (SRAM-based TCAM). In the current paper, we go a step further by supporting SR-TCAM implementation on a Virtex-5 FPGA and by getting comparable results for power consumption and latency.

To the best of our knowledge, it is among the first research work on SRAM-based TCAMs and the implementation of SRAM-based TCAM design on FPGA. Implementation on FPGA is one of the advantages over classical CAMs because currently classical CAMs cannot be implemented on FPGA.

The rest of the paper is organized as follows: Section II explains traditional CAM. Section III discusses previous work. Section IV describes architecture of SR-TCAM. Section V illustrates searching in SR-TCAM. Section VI

978-0-7695-4676-6/12 $26.00 © 2012 IEEE
DOI 10.1109/IPDPSW.2012.47

contains the hardware implementation and implementations results of SR-TCAM and section VII concludes the paper and also highlights the directions of our future work.

II. Introduction to the Traditional TCAM

Each CAM cell has its own comparison circuitry alongwith storage capacity. CAMs can be divided into two types based on its bit storage capacity: Binary CAM (BiCAM) and Ternary CAM (TCAM). BiCAM has the capability to store two logical values (0 and 1) whereas TCAM can store three states (0, 1, and X; with X is a dont care state). Thus, in TCAM, a stored key of 100X will match any of the two search keys 1000 and 1001.

TCAM consists of cells, which are arranged in the form of a two dimensional array. Cells in the same row (TCAM word) share a matchline (ML) and cells in the same column share searchlines (SLs). A conventional TCAM array is shown in Figure 1[2], which shows a 4 x 3 TCAM array consisting of 4 rows (TCAM words) and 3 columns. Each TCAM word has its corresponding ML and cells in the same column are connected to its corresponding SLs.

For a search operation, all the MLs must be pre-charged to V_{DD} and SLs must be discharged to Gnd in a conventional TCAM before the search key is to be applied. Then, the search key is applied to all SLs in parallel. If a match occurs, ML stays at its pre-charged value; otherwise it discharges to Gnd through the comparison circuitry of the TCAM cell. An ML stays at V_{DD} only, if all the cells sharing the same ML have a match condition. Otherwise, a mismatch in any cell sharing the same ML will discharge that ML to Gnd. MLs are fed to a priority encoder (PE).

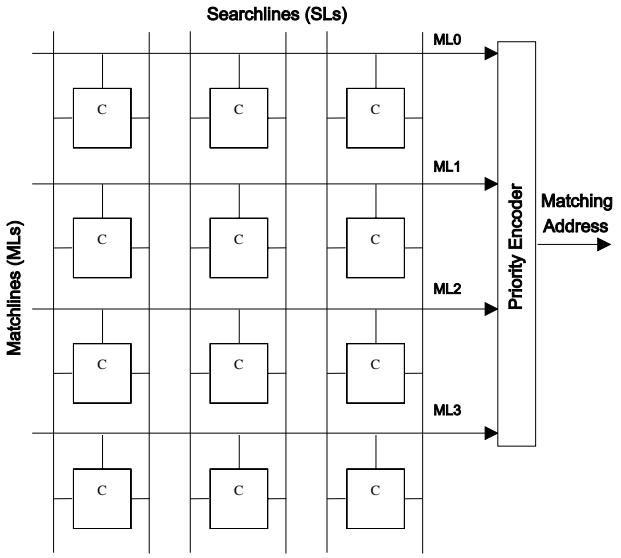

Figure 1. Traditional TCAM model showing searchlines, matchlines, memory cells, and prioirty encoder.

In BiCAM, a single match can occur, so a simple encoder is used. But in TCAM, an input word may be matched at several memory locations. Thus, a priority encoder (PE) is used to select a memory location having highest priority; conventionally, lower physical address has the highest priority. This address is then used to invoke an entry from the associated SRAM.

Figure 2 illustrates an example of search operation in classical TCAM. An input word 0111 is applied to TCAM. Two memory locations match: 1 and 2; with PE choosing the upper entry and generating the match location 1, which is then used to index its corresponding entry from the associated RAM.

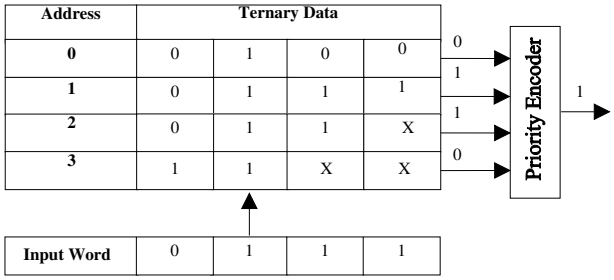

Figure 2. Example of search operation in classical TCAM.

III. Literature Overview

We surveyed CAM literature and as per best of our knowledge, we found a very few RAM-based CAM design, which are briefly discussed in this section. Many publications can be found on conventional CAMs and its variations, for example, [1], [2], [3], [4], [8], [18], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], and [30]. In contrast to these architectures, the strength of our approach is that it has been successfully implemented on FPGA, which is advantageous over classical CAMs.

In [11], authors emulate CAM with RAM, based on parallel hashing tables but this architecture is only for Binary CAM, thus lacking support for multiple matches. Furthermore, the performance of the architecture turns out to be degradable when the memory size increases and being based on hashing scheme, it suffers from collisions and bucket overflow.

A method based on conventional hashing technique to make RAM a content addressable memory has also been proposed in [12]. But this method also suffers from collisions and bucket overflow that requires additional area.

A perfect hash function is infeasible for random data. The performance of the methods in [11] and [12] intensely depends on hash function. Hash function that minimizes the probability of collision is considered good. But for a random data, which hash function is better, cannot be known in advance. In contrast the proposed SR-TCAM has the support for ternary data and also independent of data.

In [5], a RAM-based CAM has been presented, specifically targeting its application to ATM communication sys-

tems but it has some serious shortcomings. Size of this scheme basically depends on the number of bits in a pattern (x). The required memory size would be 2^x bits. Size will increase exponentially with the increase in pattern size. For instance, 32 bit IP address needs a 4 GB of RAM and a 72 bit pattern needs 4294967296 TB of RAM, which is practically infeasible in terms of area, cost, and power consumption. It might be a good alternative for CAM in ATM communication systems but cannot be used in applications, which need area, cost, and power as efficiency metrics. In contrast, SR-TCAM supports arbitrarily large bit patterns.

The alternative for TCAM presented in US patent [13] also suffers from certain disadvantages. It is a hybrid approach in the sense that it integrates CAM and RAM to get overall CAM functionality, thus inheriting the instinctive disadvantages of CAM. In this scheme, CAM table is divided in to groups using some distinguishing bits in CAM entries in such a way that only one entry in each group can possibly match any given input. In typical TCAM applications, data is totally random, thus grouping would be a very tedious job by finding distinguishing bits in CAM entries. This scheme is suitable for Internet packet classification as claimed by the patent itself and hence, not generic in its nature. On the other hand, SR-TCAM is generic and can be used in any TCAM application.

Approach presented in [14] also suffers from size disadvantage, only works on data arranged in ascending order, does not consider storage of original addresses, and does not have an appropriate memory partitioning scheme. But in contrast, SR-TCAM supports an arbitrary large bit pattern, considers the storage of original addresses, and has appropriate partitioning scheme for achieving optimized TCAM design.

IV. Architecture of SR-TCAM

To the best of our knowledge, it is among the first research work on SRAM-based TCAMs and implementation on FPGA. SR-TCAM is an extension of [10] and [17] in terms of FPGA implementation and actually measuring power consumption and latency, which have not been described in [10] and [17].

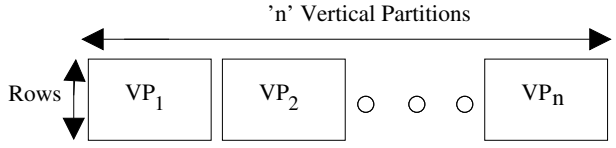

Figure 3. Conceptual view of vertical partitioning

SR-TCAM vertically partitions conventional TCAM table column-wise into n number of TCAM sub-tables, which are then processed to be stored in their corresponding SRAM blocks. These SRAM-blocks are termed as address position tables (APTs) and bit position tables (BPTs).

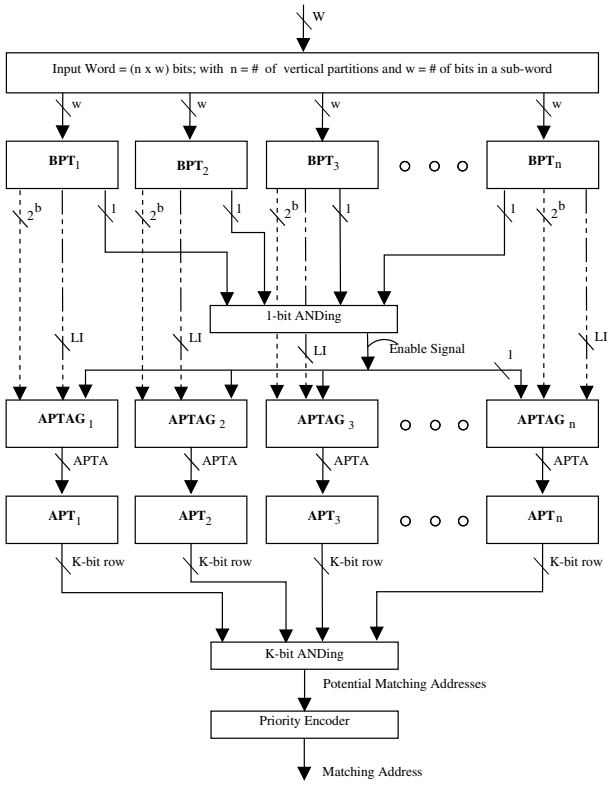

Figure 4. Architecture of SR-TCAM

Vertical partitioning implies that a TCAM word of width W bits are divided into n sub-words, each of which is of width w bits. A conceptual view of vertical partitioning is illustrated in Figure 3 where a row corresponds to a TCAM word.

Main components of memory architecture of SR-TCAM, shown in Figure 4, include n BPTs, n APTs, n APT address (APTA) generators, priority encoder (PE), and ANDing operation. Each vertical partition has its corresponding BPT, APTA generator, and APT.

Maximum possible combinations of w bits are 2^w where each combination represents a sub-word. Our aim is to map 2^w sub-words to 2^w bits such that each sub-word would be represented by a single bit in its corresponding memory.

In BPT, 2^w bits of memory are arranged into 2^{w-b} rows; with each row has 2^b bits, which can be two or more. Each row is supplemented with a value called last index (LI) of length w+1 bits. The w-b high order bits of input sub-word are used to select a particular row in BPT, thus acting as an address (BPTA). The b low order bits, termed as bit position indicator (BPI), of the input sub-word are used to indicate a particular bit position in the row selected. If the bit position indicated by BPI is high, then it means that the input sub-word is present, otherwise not. Conceptual view of BPT is shown in Figure 5.

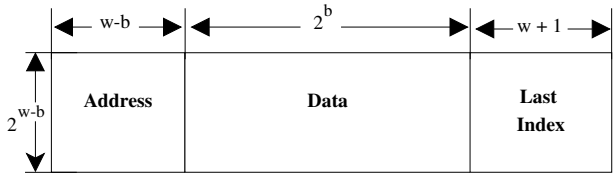

Figure 5. Conceptual view of BPT

APTA generator generates an address referred to APTA, which is used to index a row in its corresponding APT. APTA generator contains 1‘s counter and adder. 1‘s counter counts number of 1‘s in the selected row of BPT up to the indicated bit position inclusive and then forwards this information to adder. Adder then adds the output of the 1‘s counter and LI of the selected row. SR.-TCAM borrows the concept of BPT and APTA generator from US patent [14].

Size of APT is 2^w * K where 2^w represents number of rows and K is the number of bits in each row where each bit represents an address position. This address position corresponds to its original address. A conceptual view of APT is illustrated in Figure 6.

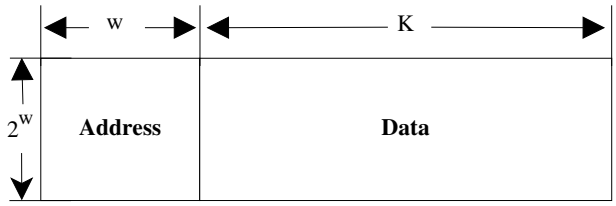

Figure 6. Conceptual view of APT

V. Searching in SR-TCAM

During searching, an input word is applied and, if exits, its matching address (MA) is sent to output port. SR-TCAM accomplishes search operation as per steps depicted in the flow chart of Figure 7. These steps are explained below:

- **Step 1:** an input word is applied to SR-TCAM.
- **Step 2:** the applied word is partitioned into n number of sub-words. These sub-words are then applied to their corresponding BPTs in parallel.
- **Step 3:** a sub-word is then divided into two portions: BPTA and BPI. Step 3 occurs for all sub-words in parallel.
- **Step 4:** bit position in BPT indicated by BPI in the row selected by BPTA is read out. If the read out bit is high (1), it shows that input sub-word is considered present (otherwise, not) but at which address, still unknown. Step 4 is also performed in parallel for all BPTs.
- **Step 5:** the read out bits from all BPTs in step 4 are ANDed. Step 5 corresponds to 1-bit ANDing in Figure 4. The result of this 1-bit ANDing operation specifies whether the searching effort is to be continued or to be

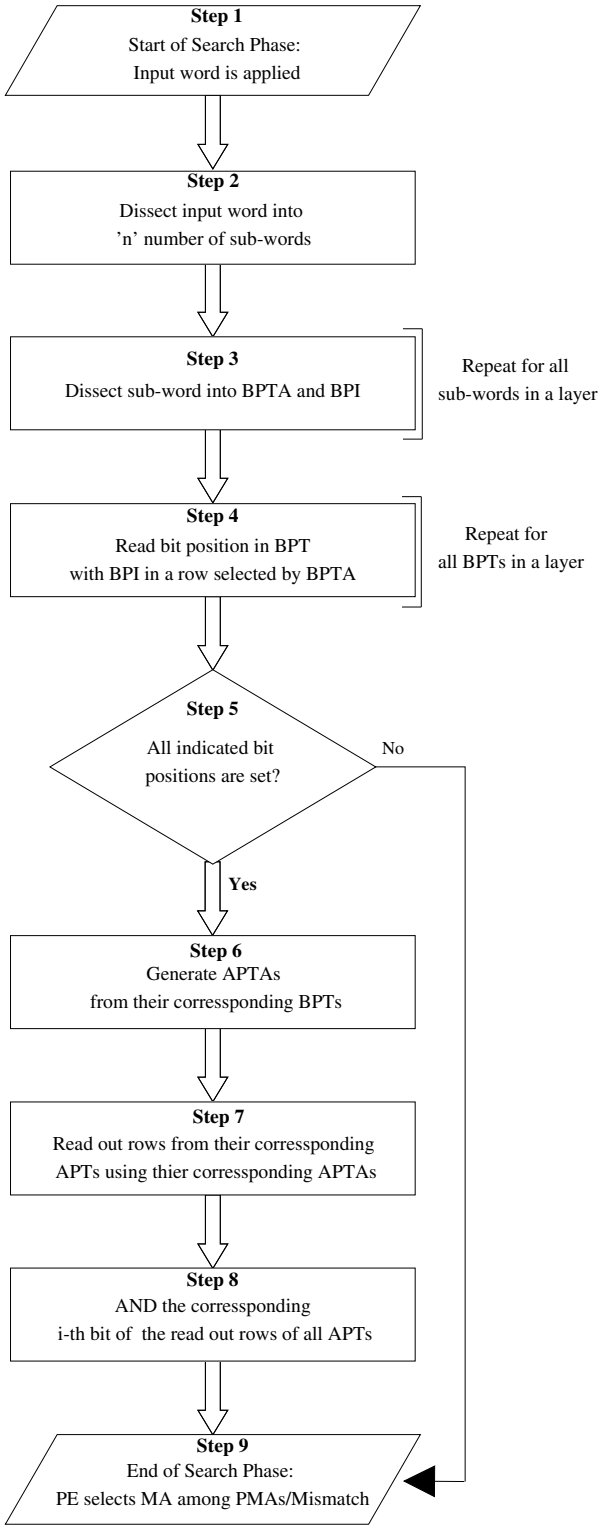

Figure 7. Flowchart showing search in SR-TCAM

stopped. If the 1-bit ANDing results in a low signal, it means that mismatch has occurred and shows end of search phase, otherwise the proposed TCAM sustains search operation and enters step 6.

- **Step 6:** APTA generator computes APTA by summing the number of ones (1) in the selected row up to the bit position inclusive indicated by BPI and LI of the selected row of BPT. Step 6 is also carried out in parallel for the computation of all the APTAs.
- **Step 7:** APTAs, from step 6, read out rows from their corresponding APTs simultaneously.
- **Step 8:** It shows ANDing operation, which corresponds to K-bit ANDing in Figure 4. The corresponding i^{th} bits of all rows read out in step 7 are ANDed where $0 \leq i \leq K-1$. The address positions that remain at high level (1) after ANDing are considered potential matching addresses (PMAs).
- **Step 9:** It is the last step. A PE selects MA among PMAs, if exists, otherwise a mismatch is signaled.

A mismatch of an input word can be detected at two places as shown in Figure 7. First, when any of the bit position pointed by its corresponding BPI is not set in its respective BPT as discussed in step 5 and second, when all the address positions are low (0) after ANDing in step 8.

When bit positions indicated by BPIs in all BPTs are set as mentioned in step 5 of Figure 7, then this condition permits the searching effort to be continued. At this point, it cannot be decided that the input word is present. Whenever this case occurs, the match/mismatch is decided by the K-bit ANDing operation of step 8.

VI. FPGA Implementation of SR-TCAM

A sample design of 512 x 36 SR-TCAM has been implemented and verified on reconfigurable FPGA platform. We have used Xilinx Virtex-5 FPGA board for porting the implemented design.

The sample design has been synthesized using Xilinx synthesizer and place & route tools. The functionality of the design has been verified extensively by using ModelSim SE verification tool over a large set of test cases. Implementation of the design follows Figure 4.

Input word of 36 bits has been first divided into three sub-words, each of which is of 12 bits. Each sub-word has then been applied as an address to its corresponding memory module, BPT, which is of size 512 x 21 bits. For the design sample, a total of three BPTs are required. Size of each BPT has been decided by sub-dividing the sub-word into two parts, 9 bits and 3 bits, thus needing address space of 2^9 = 512 and word size of 2^3+13 (LI) = 21bits. Data accessed from a BPT has been processed using its corresponding APTAG to generate APTA for its corresponding APT.

In our sample SR-TCAM design, three APTs have been used, and the size of each APT is 4096 x 512 bits. Read out rows from all APTs have then been ANDed bit-by-bit to get MA using PE.

Since, the design of a large PE is itself have several trade-offs, we are not focusing on this area. Instead, we have taken a simple implementation of PE just to have functional evaluation first. Moreover, the entire design can be very much optimized in terms of delay by improved implementation of PE. Below we will see the impact of the inclusion of the priority encoder in the design. We have shown the design numbers with and without PE. For better results, user can opt of many available PE design techniques to have optimized design numbers.

Table II shows complete implementation details of the sample design. We have used two synthesis optimizations using BRAMs on FPGA. The FPGA board (xc5vlx220-ff1760-2) that we have used has 192 BRAMs available on it. Each BRAM on Virtex-5 can have maximum dimensions of 1024 x 36 and can be configured in many possible ways.

In case of first design (Design 1), by using synthesis option BRAM-AUTO, it has used 192 BRAMs for all three APTs and has created three BPTs using distributed BRAMs.

In case of second design (Design 2), by using synthesis option BRAM = BLOCK_POWER2, it has used only 174 BRAMs for APTs and 3 BRAMs for BPTs. Except from the numbers of BRAMs used, both designs have also some trade-off in terms of amount of LUTs and delays. Thus, user can select design parameters that are suitable for his application.

The power data has been measured using Xilinx X-Power [31] by using activities for search operations of SR-TCAM. We have computed power consumption by taking the average of 1000 search operations with 100 MHz clock speed to have better power estimation.

Table II
Implementation detials of the sameple design

Design	LUTs	FFs	BRAMs	T-period (ns)	Average Power Consumption (mW)
Design 1	1135	1144	192	8.560	2.40
Design 2	2199	2208	177	9.738	2.14
Design 1 with PE	1966	1975	192	48.721	2.16
Design 2 with PE	3160	3168	177	45.087	2.11

VII. Conclusion

In this paper, we present an implementation of 512 x 36 SR-TCAM on Xilinx Vertix-5 FPGA along-with its analysis in terms of power consumption and latency. Implementation on FPGA is the strength of SR-TCAM.

Power consumption and latency for different synthesis optimizations have been tabulated. Power and latency with

PE and without PE have also been analyzed to validate feasibility and functionality of SR-TCAM. SR-TCAM achieves comparable searching operation in two clock cycles. Noted that SR-TCAM targets large capacities TCAMs whereas this capability is lacked by conventional ones.

Other attractive points of SR-TCAM are that its performance is independent of data and can support arbitrarily large bit pattern. We hope that with successful FPGA implementation, the current design can lead us to a new CAM world. We further hope that SRAM future development can give us better values for the current design.

Our future work includes taking different TCAM dimensions and taking various numbers for vertical partitions to get a trade-off between the value for n and power consumption along with latency. We are also trying to find more methods to configure SRAM to get over all TCAM functionality.

REFERENCES

[1] N. Mohan, W. Fung, D. Wright, and M. Sachdev Design Techniques and Test Methodology for Low-Power TCAMs IEEE Transactions on Very Large Integration (VLSI) Systems, vol. 14, no.3, June 2006.

[2] K. Pagiamtzis and A. Sheikholeslami, Content-Addressable Memory (CAM) Circuits and Architectures: A Tutorial and Survey, IEEE Journal of Solid-State Circuits, vol. 41, no. 3, pp.712-727, March 2006.

[3] D.E. Taylor, Survey Taxonomy of Packet Classification Techniques, in Proc. WUCSE-2004-24, Tech. Report, May 2003.

[4] Fang Yu, T. V. Lakshman, Martin Austin Motoyama, and Randy H. Katz, Efficient Multimatch Packet Classification for Network Security Applications, IEEE Journal on selected areas in Communication, vol.24, no. 10, Oct. 2006.

[5] Stamatios V. Kartalopoulos, An Associative RAM-Based CAM and its Application to Broad-Band Communication Systems, IEEE Transactions on Neural Networks, vol. 9, No. 5, pages: 1036-1041, Sep. 1998

[6] Patrick Mahoney, Yvon Savaria, Guy Bois, and Patrice Plante, Performance Characterization for the Implementation of Content Addressable Memories Based on Parallel Hashing Memories, P. Stenstrom (ED.): Transactions on HiPEAC II, LNCS 5470, pp. 307-325,2009.

[7] Renesas CAM ASSP Series. http://www.renesas.com.

[8] F. Zane, G. J. Narlikar, and A. Basu.CoolCAMs: Powere fficient TCAMs for forwarding engines. In Proc. INFOCOM 03, pages 4252.

[9] CACTI. http://www.hpl.hp.com/personal/norman jouppi/cacti4.html.

[10] Zahid Ullah, SRAM-based Ternary Content Addressable Memory, Master Thesis, Hanyang University, South Korea, Feb. 2010.

[11] Mahoney, P., Savaria. Y., Bois, G., and Plante, P., Parallel Hashing Memories: an Alternative to Content Addressable Memories, 3rd International IEEE-NEWCAS Conference, pages: 223-226, June 2005.

[12] Sangyeun Cho, Martin, J. R., Ruibin Xu, Hammoud, M. H., and Melhem, R., CA-RAM: A High-Performance Memory Substrate for Search-Intensive Applications, In Proceedings of Performance Analysis of Systems & Software, 2007.ISPASS 2007, IEEE International Symposium, pp.230-241, April, 2007.

[13] Madian Somasundaram, Memory and Power Efficient Mechanism for Fast Table, Lookup, US Patent, No. US 7,296,113,B2, Nov. 13,2007

[14] Madian Somasundaram, Circuit to Generate a Sequential Index for an Input Number in a Pre-defined List of Numbers, US Patent, No. US 7155563 B1, Dec. 26, 2006.

[15] Cypress Sync SRAMs. http://www.cypress.com.

[16] SAMSUNG High Speed SRAMs. http://www.samsung.com..

[17] Zahid Ullah and Sanghyeon Baeg, Vertically Partitioned SRAM-based Ternary Content Addressable Memory, to be appeared in Elsevier Journal, Procedia Engineering, 2011.

[18] Young-Deok Kim, Hyun-Seok Ahn, Suhwan Kim, and Deog-Kyoon Jeong, A High-Speed Range-Matching TCAM for Storage-Efficient Packet Classification, IEEE Transactions on Circuits and Systems-1, vol. 56, no. 6, June 2009.

[19] Byung-Do Yang, Yong-Kyu Lee, Si-Woo Sung, Jae-Joong Min, Jae-Mun Oh, and Hyeong-Ju Kang, A Low Power Content Addressable Memory Using Low Swing Search Lines, IEEE Transactions on Circuits and Systems-1, vol. 58, no. 12, December 2011.

[20] N. Mohan, W. Fung, D. Wright, and M. Sachdev, A Low-Power Ternary CAM With Positive-Feedback Matchline Sense Amplifiers,, IEEE Transactions on Circuits and Systems-I, vol. 56, no. 3, March 2009

[21] Ahmed Youssef, Zhen Yang, Mohab Anis, Shawki Areibi, Anthony Vannelli,, and Mohamed Elmasry, A Power-Efficient Multipin ILP-Based Routing Technique , IEEE Transactions on Circuits and Systems-I, vol. 57, no. 1, January 2010.

[22] Yen-Jen Chang,A High-Performance and Energy-Efficient TCAM Design for IP Address Lookup, IEEE Transactions on Circuits and Systems-II, vol. 56, no. 6, June 2009.

[23] Yen-Jen Chang,Using the Dynamic Power Source Technique to Reduce TCAM Leakage Power, IEEE Transactions on Circuits and Systems-II, vol. 57, no. 11, November 2010.

[24] K. Pagiamtzi and A. Sheikholeslami, A Low-Power Content-Addressable Memory (CAM) Using Pipelined Hierarchical Search Scheme, IEEE Journal of Solid State Circuits, vol. 39, no. 9, September 2004

[25] Shanq-Jang Ruan, Chi-Yu Wu, and Jui-Yuan Hsieh, Low Power Design of Precomputation-Based Content Addressable Memory, IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 16, no. 3, March 2008.

[26] Wencheng Lu and Sartaj Sahni Low-Power TCAMs for Very Large Forwarding Tables, IEEE Transactions on Networking, vol. 18, no. 3, June 2010.

[27] H. Noda et al. A Cost-Efficient High-Performance Dynamic TCAM With Pipelined Hierarchical Searching and Shift Redundancy Architecture, IEEE J. Solid-State Circuits, 40(1):245253, Jan. 2005.

[28] I. Arsovski, T. Chandler, and A. Sheikholeslami, A ternary content addressable memory (TCAM) based on 4T static storage and including a current-race sensing scheme, IEEE J. Solid-State Circuits, vol. 38, no. 1, pp. 155158, Jan. 2003.

[29] C. A. Zukowski and S.-Y. Wang, Use of selective pre-charge for low power content-addressable memories, in Proc. IEEE Int. Symp. Circuits Syst. (ISCAS), vol. 3, 1997, pp. 17881791.

[30] K. Pagiamtzis and A. Sheikholeslami, A low-power content-addressable memory (CAM) using pipelined hierarchical search scheme, IEEE J. Solid-State Circuits, vol. 39, no. 9, pp. 15121519, Sep. 2004.

[31] Xilinx X-Power. http://www.xilinx.com